

BINARY TREE LAYOUTS WITH ADEQUATE I/O SUPPORT

Nian-Feng Tzeng and Sourav Bhattacharya

The Center for Advanced Computer Studies
The University of Southwestern Louisiana
P. O. Box 44330
Lafayette, LA 70504

Abstract — The binary tree-based parallel system is suitable for VLSI/WSI implementation due to its simple and regular interconnection pattern among processors. The support of adequate I/O bandwidth in a large parallel system is essential as otherwise, the system speed gets limited eventually by the I/O operations. In a tree-based system, I/O operations are efficiently carried out through the root and the leaf nodes. This paper presents a way to systematically generate a tree layout with a sufficient number of leaf nodes on the boundary to support required I/O bandwidth, while maintaining a low area increase and moderate growth in maximum link length. This technique will be particularly useful for large sized binary tree layouts.

1. Introduction

Recent advances in VLSI/WSI technology make it possible to construct a cost-effective parallel system comprising many processing elements. Each processing element may have its own local memory and has either independent control or global control. A task can be divided into subtasks and distributed into processors for execution, achieving high concurrency. The binary tree-based parallel system has been studied intensively, because many different concurrent algorithms can be mapped onto the binary tree architecture efficiently [1]. In addition, the binary tree requires only three ports per node and has simple and regular interconnection among nodes, making it fairly suitable for VLSI/WSI implementation.

For a large parallel system, the support of efficient I/O operations is essential. If the designed I/O fails to provide adequate bandwidth for problems under execution in the system, even if the computation power of the system can solve problems much faster, the system speed is ultimately limited by the I/O bottleneck. The importance of balancing I/O bandwidth and computational power has been pointed out by Kung [2]. I/O processors are used in parallel system for transferring data between processing elements and the outside world as well as the host. Specifically, they distribute program codes and data before computation starts and collect the results after computation is finished. In a binary tree-based system, I/O processors are often connected to the root and leaf nodes where data/program distribution and result collection can be efficiently carried out. We assume here that I/O operations are done only through the root and leaf nodes.

A single wafer of size 4 to 8 inches can accommodate a binary tree with 14 levels [3], and probably more in the future. To lay out the binary tree on a chip has been well treated [4, 5, 6]. Every layout allows the root to have direct access to the external. The "H-tree" layout [5] yields a more compact result, but only a portion of the leaf nodes in the layout are on the boundaries and can support I/O operations. Moreover, the ratio of boundary leaf nodes to total leaf nodes decreases rapidly as the tree size increases. For example, a 7-level tree has roughly 52% leaf nodes on the boundaries, whereas a 12-level tree has less than 10% boundary leaf nodes. Suppose each boundary leaf node is connected to an I/O processor, then, this low number of boundary leaf nodes may not be sufficient to support the needed I/O bandwidth. On the other hand, to achieve a layout with 100% leaf nodes on the boundaries requires considerably larger area [4]. Furthermore, the maximum link length in this layout is significantly bigger, which may lead to a much lower system performance because the clock

This work was supported in part by the NSF under Grant MIP-8807761 and by the Louisiana Board of Regents' R&D Program Component under Grant 86-USL(2)-127-03.

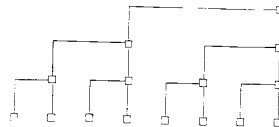


Fig. 1. A "standard" tree layout for a 4-level tree.

speed of a system is normally determined by the worst case communication delay in the system. It may not always require every leaf node to have direct access to an I/O processor, and therefore a layout of this kind is not always desirable. Our primary motivation stems from the need to generate a compact binary tree layout with adequate I/O support and good performance. This is particularly important when the system size is large.

In this article, we address a systematic way to lay out binary trees based on the requirement that a certain percentage of leaf nodes has to be on the boundaries for providing the required I/O bandwidth. Every layout so generated is planar and on a rectangular grid. The binary tree of interest here is a complete one and contains thousands of nodes. A node in a layout is assumed to take one unit width, so is a link between two connected nodes. Nodes are separated by at least unit distance. The *area* of a layout is the area of the smallest rectangle which contains all the links and nodes. The *length* of a link is the distance between its two end points, with the distance between any two neighboring grid points being one. For many practical situations, both area and the maximum link length in our resulting layout are increased only moderately in comparison with those in the "H-tree" layout. This layout is also shown to be better than the one obtained by simply adding extra tracks to a typical "H-tree" layout so as to bring internal leaf nodes to the boundaries.

2. Standard Tree Layout and H-Tree Layout

The usual layout of a binary tree, termed the "standard" tree layout, with 4 levels (which contains 2^3 leaf nodes) is illustrated in Fig. 1. It is obvious that such a layout takes area $m \times 2^{m-1}$ for a tree with m levels. The maximum link length between two nodes in this layout is of order 2^{m-2} , a big value for a large tree. In this case, all the leaf nodes are placed on the boundary, to which I/O ports are connected. It has been shown that the minimum layout area for an m -level binary tree constructed this way is of order exactly $m \times 2^{m-1}$, provided that the chip boundary is convex, a practical situation [4].

An alternative layout for binary trees, dubbed the "H-tree" layout, was proposed by Horowitz and Zorat [5]. The "H-tree" layout of a 7-level tree is depicted in Fig. 2, where the root is numbered 1, its two children are 2 and 3, etc. It yields a more compact layout with the maximum link length reduced drastically. Generally, the area required for an m -level tree, A_m^H , is

$$A_m^H = (2^{\frac{m+2}{2}} - 1) \times (2^{\frac{m}{2}} - 1) = 2^{m+1} - 3 \times 2^{\frac{m}{2}} + 1 \quad (1)$$

for an even m , and

$$A_m^H = (2^{\frac{m+1}{2}} - 1)^2 = 2^{m+1} - 2^{\frac{m+3}{2}} + 1 \quad (2)$$

for an odd m . Therefore, the area is of the order $O(2^{m+1})$. The length of the longest link, L_m^H , in this layout, is $2^{\lfloor m/2 \rfloor - 1}$ (where

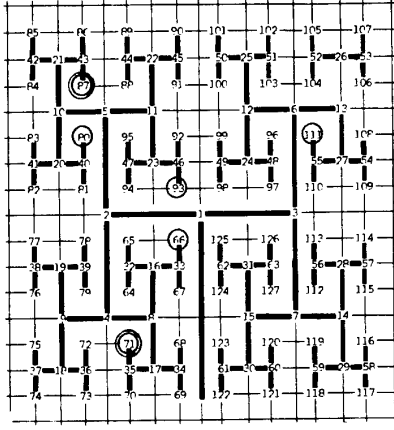


Fig. 2. The "H-tree" layout of a 7-level tree (from [5]).

the longest link means the longest "internal" link, excluding the link between the root and the external).

The potential deficiency associated with the "H-tree" layout lies in that only a fraction of leaf nodes are on the boundaries, as can be seen in Fig. 2. If we treat those internal leaf nodes which can be connected to the boundary through unused channel segments as boundary leaf nodes also, called *extended* boundary leaf nodes, such as nodes 87, 71, etc. Then, the number of boundary leaf nodes in an m -level tree layout, B_m^H , is expressed by

$$B_m^H = 3 \times 2^{\frac{m}{2}} - 4 + \delta_1, \quad (3)$$

for an even m , and

$$B_m^H = 2^{\frac{m+3}{2}} - 4 + \delta_1, \quad (4)$$

for an odd m , where δ_1 is due to the extended boundary leaf nodes and equals $2m - 9$ (or 0) for $m \geq 5$ (or otherwise). It is to be noted that B_m^H is $O(2^{m/2})$ always. The ratio of B_m^H to the total number of leaf nodes, 2^{m-1} , decreases rapidly as m grows. It might soon reach the point where the boundary leaf nodes are unable to support the needed I/O bandwidth. For example, a 14-level "H-tree" layout has only 399 leaf nodes on the boundaries, which account for less than 5% of the 8192 total leaf nodes.

A trivial way to increase the number of boundary leaf nodes is by introducing additional tracks in between the adjacent rows and/or columns in an "H-tree" layout so as to bring the internal leaf nodes to the boundaries. By adding a horizontal track and a vertical track, for example, we may bring four internal leaf nodes to the boundaries, namely, nodes 111, 80, 93, and 66 as marked (see Fig. 2), provided that crossings of two wires are allowed. In this case, the area is increased from 15×15 to 16×16 and the length of the longest link, from 8 to 9. It is possible to add as many tracks as needed so as to bring enough internal leaf nodes to the boundaries, with both the area and the maximum link length increased accordingly.

Now, suppose that e additional leaf nodes in a typical "H-tree" layout must be brought to the boundaries to provide sufficient I/O support, then at least $e/2$ tracks must be added to the layout (since each additional track can bring at most two internal leaf nodes out). Considering an m -level "H-tree" layout, for an even m , the layout is rectangular with one size about twice the size of the other one (say, the wider size is along the vertical direction); whereas for an odd m , the layout is square. Let us assume that the added tracks are distributed uniformly across the layout to avoid possibly creating any "very long" link. As a result, one third (or half) of the $e/2$ tracks would run vertically and the rest, horizontally, for an even (or odd) m . The expressions for the area taken by the resulting layout can be easily derived from Eqs. (1) and (2) as

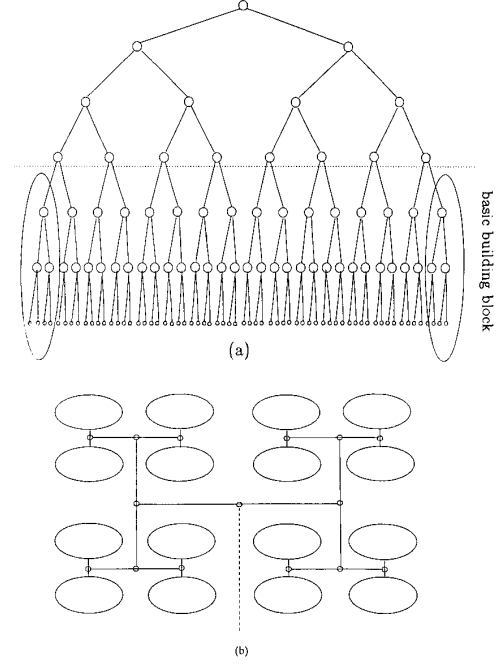


Fig. 3. (a) A 7-level binary tree with its lowest three levels as basic building blocks. (b) The resulting layout of (a).

(The tree levels are numbered upwards, with the lowest level being level 1. Only two of the basic building blocks in (a) are marked.)

$$(2^{\frac{m+2}{2}} - 1 + \frac{e}{3}) \times (2^{\frac{m}{2}} - 1 + \frac{e}{6})$$

for an even m , and

$$(2^{\frac{m+1}{2}} - 1 + \frac{e}{4})^2$$

for an odd m . The area, therefore, is of the order $O((2^{m/2} + e)^2)$. The length of the longest link in the layout becomes $(2^{(m-2)/2} + \frac{e}{8(1-2^{-m/2})})$ for an even m , and $(2^{(m-1)/2} - 1 + \frac{e}{4})$ for an odd m . This layout result will be compared to that obtained by using our proposed layout algorithm in the next section.

3. Proposed Tree Layout

To increase the number of boundary leaf nodes in a layout, we may treat subtrees consisting of lowest several tree levels as basic building blocks and follow the "H-tree" layout procedure to construct the whole tree layout. This idea is best illustrated by an example. Suppose a 7-level binary tree is to be laid out and the basic building block involves its lowest three levels, as indicated in Fig. 3(a). The resulting layout of the algorithm is shown in Fig. 3(b), where an ellipse represents a basic building block (which can be laid out, according to the "standard" tree layout, with all leaf nodes in it placed along one side, as illustrated in Fig. 1). In this case, all the 64 leaf nodes are on the boundaries, in contrast to a pure "H-tree" layout, which has only about 52% boundary leaf nodes. For a large tree, subtrees involving more levels may be utilized to have more leaf nodes on the boundaries in a resulting layout.

The layout of basic building blocks determines the area required for the whole tree layout. Following the "standard" tree layout [4], a k -level subtree takes area of order $k \times 2^{k-1}$. A tree layout with its "cost" minimized has been considered by Fischer and Paterson [7], where the layout cost is the sum of all link costs, each of which is defined as link length times link weight. Com-



Fig. 4. Our layout strategy for basic building blocks to minimize area.

pared with Fig. 1, this layout takes exactly the same area. We use a slightly different layout strategy for the basic building block. Unlike in [7], our goal here is to minimize the area, so we let every two consecutive levels above the lowest two levels share the same row, as depicted in Fig. 4. (The second lowest level cannot share a row with the third lowest level due to insufficient room.) It effectively leads to a constant factor improvement in area. Notice that the maximum link length in the layout of basic building blocks now is longer than that in [7]. However, the longest link of the whole tree layout will be the link connected to the root, not the longest link in a basic building block, as long as $(m-k) \geq 1$ (where m and k are respectively the binary tree and the subtree levels; we shall use these symbols throughout this paper).

It is clear that the layout so generated is a compromise between a pure "H-tree" layout and a pure "standard" tree layout. For a given binary tree, its layout area required increases as the number of boundary leaf nodes grows, realized by treating larger subtrees as basic building blocks. At the same time, the maximum link length may also be extended in the layout.

3.1. Layout Algorithm

The procedure for generating the layout can be designed as a recursive procedure. To generate the layout of an m -level tree, the procedure invokes the same call for two $(m-1)$ -level subtrees, and then merges them along with the allocation of the current root. The process of merging requires the coordinates of the layout of one of the two subtrees to be shifted either along the x -axis or along the y -axis, depending on the orientation in which the two subtrees coalesce to form the bigger tree. The lower left corner of the layout grid is given coordinate $(0, 0)$. When m is small enough to be laid out as a subtree, it is directly laid out. Conceptually, this procedure involves the following five steps.

- Step 1: if $m \leq k$ then do direct-layout, and return;
- Step 2: get a layout for an $(m-1)$ -level subtree.
- Step 3: shift this layout up (or right) by the y (or x) size of the layout, when two such layouts join one above another (or side by side).
- Step 4: get another layout for an $(m-1)$ -level subtree.
- Step 5: place the root at the mid-position of the two layouts.

For a given subtree with k levels, the procedure for directly laying out produces the coordinates of every node in the subtree based on our layout strategy for minimizing the area taken, as illustrated in Fig. 4. By observing several cases, we find out that those 2^{k-h} nodes at level h will be placed at coordinate (x, y) , where $y = \lceil (k-2)/2 \rceil - \epsilon$, with $\epsilon = \lceil (h-2)/2 \rceil$ (or -1) as $h \geq 2$ (or $h=1$), and coordinate x for a node at position j in the level (with the leftmost one called position 0), $0 \leq j \leq 2^{k-h}-1$, is $(j \cdot 2^h + P[h][1])$ or $(j \cdot 2^h + P[h][2])$, where P is a 2-dimensional array storing a fixed coordinate pattern, characterizing our layout style, as supplied below

$\langle 0,1 \rangle, \langle 1,2 \rangle, \langle 2,5 \rangle, \langle 4,11 \rangle, \langle 11,20 \rangle, \langle 15,47 \rangle, \langle 47,79 \rangle, \langle 63,191 \rangle, \dots$

3.2. Result and Comparison

Area required for the layout generated by our algorithm. $A_{m,k}^G$, is

$$A_{m,k}^G = (2^{\frac{m+k-2}{2}} + 2^{\frac{m-k}{2}} - 1) \times ((3+\eta)2^{\frac{m-k}{2}} - 1) \quad (5)$$

when $(m-k)$ is even, and

$$A_{m,k}^G = (2^{\frac{m+k-3}{2}} + 2^{\frac{m-k-1}{2}} - 1) \times ((3+\eta)2^{\frac{m-k}{2}} - 1) \quad (6)$$

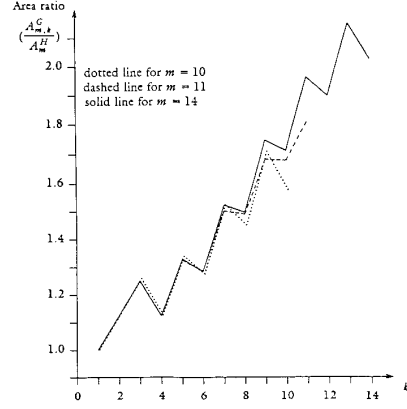


Fig. 5. Area ratio versus k for various tree sizes.

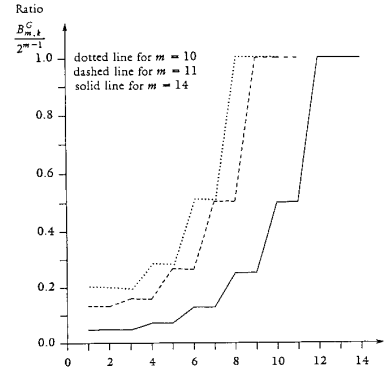


Fig. 6. Ratio of boundary leaf nodes to total leaf nodes versus k for various tree sizes.

when $(m-k)$ is odd, where $\eta = \lceil (k-2)/2 \rceil$ (or -1) for $k > 1$ (or $k=1$). Notice that Eqs. (5) and (6) are reduced respectively to Eqs. (1) and (2) when k is 1, as expected. Under both the situations the area required is of the order $O(2^{(m+k)/2} \times k 2^{(m-k)/2})$. The maximum link length in the layout, $L_{m,k}^G$, is $(2^{\frac{m+k-6}{2}} + 2^{\frac{m-k-4}{2}})$ when $(m-k)$ is even, and is the larger of $(2^{\frac{m+k-7}{2}} + 2^{\frac{m-k-5}{2}})$ and $(3+\eta)2^{\frac{m-k-3}{2}}$ when $(m-k)$ is odd. The total number of boundary leaf nodes, $B_{m,k}^G$, including the extended boundary leaf nodes, for $m > k$ is given by

$$B_{m,k}^G = 2^{\frac{m+k}{2}} + 2^{\frac{m-k+2}{2}} - 4 + \delta_2, \quad (7)$$

when $(m-k)$ is even, and

$$B_{m,k}^G = 2^{\frac{m+k-1}{2}} + 2^{\frac{m-k+3}{2}} - 4 + \delta_2, \quad (8)$$

when $(m-k)$ is odd, where δ_2 is due to extended boundary leaf nodes and equals $2m-2k-7$ (or 0) for $m-k \geq 4$ (or otherwise). $B_{m,m}^G$ is equal to 2^{m-1} . Again, when k is 1, Eq. (7) becomes Eq. (4) and Eq. (8) becomes Eq. (3). It is to be noted that $B_{m,k}^G$ is of the order $O(2^{(m+k)/2})$.

We plot the ratio of $B_{m,k}^G$ to total leaf nodes versus k for various tree sizes in Fig. 5. Fig. 6 gives the ratio of $A_{m,k}^G$ to A_m^H as a function of k for various m , and Fig. 7 illustrates the ratio of $L_{m,k}^G$ to L_m^H versus k . The cases of $k=1$ and 14 in these figures denote respectively the pure "H-tree" layout and the pure "standard" tree layout (which follows the layout strategy of minimizing area as shown in Fig. 4). Compared with the pure "H-tree" layout, the "standard" tree layout requires more than twice area when m

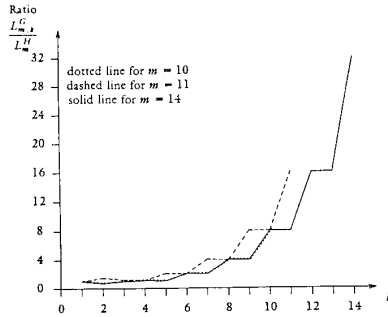


Fig. 7. The maximum link length ratio versus k for various tree sizes.

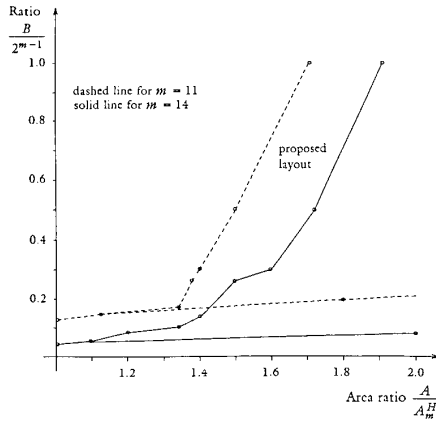


Fig. 8. Comparison of two types of layouts in terms of the area increase ratio.

$m=14$ (see Fig. 6) and has its maximum link length 32 times as big (which might be translated into a substantially slow system clock speed). It can be seen from Fig. 5 that the percentage of boundary leaf nodes for $m=14$ is increased from less than 5% to more than 25% as k varies from 1 to 8. The price for this is the increase in layout area by slightly less than 50% (see Fig. 6), and in the maximum link length by about 300% (see Fig. 7). It is interesting to observe from the three figures that when m is even, an even k yields a layout with noticeably less area in contrast to $k-1$, but with a bigger maximum link length and fewer boundary leaf nodes. For any m and a given number of boundary leaf nodes required in a layout, we can find out an appropriate k and follow the algorithm provided above to generate the layout.

The node utilization in this kind of layouts can be easily computed as the total number of nodes in an m -height tree divided by the area $A_{m,k}^G$. After substituting the values, it is noted that both for the cases of $(m-k)$ odd and even, the node utilization is inversely proportional to $(3+\eta)$. Since η is proportional to k , we can expect the node utilization to vary with respect to k following a hyperbolic pattern, and the even-height subtree becomes more efficient from the node utilization point of view compared to its immediate previous odd-height subtree. This is due to the fact that our subtree-layout style (see Fig. 4) allows every two levels, above the lowest two levels, share the same row.

To compare this layout with the one produced by adding extra tracks uniformly across an "H-tree" layout as described in the previous section, we demonstrate the ratio of boundary leaf nodes to the total leaf nodes as a function of the area increase for both of the layouts in Fig. 8. The layout generated by our algorithm is clearly better in each tree size: for any given area increase,

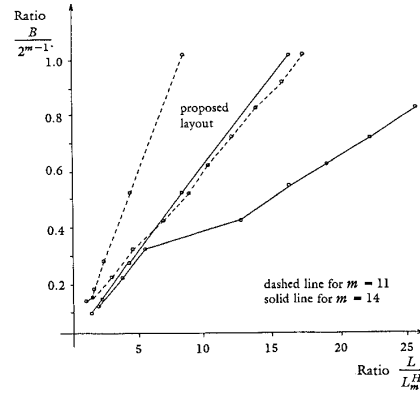


Fig. 9. Comparison of two types of layouts in terms of the maximum link length ratio.

a higher percentage of leaf nodes in our layout is on the boundary, and the difference becomes bigger as the incremental amount grows. A similar conclusion can be drawn when the maximum link length is considered, as depicted in Fig. 9.

4. Concluding Remarks

We have proposed an algorithm for generating binary tree layouts with various numbers of leaf nodes on the boundaries for any given tree size. The algorithm is particularly useful for a large area binary tree where the number of boundary leaf nodes in the "H-tree" layout is insufficient to support necessary I/O bandwidth and more leaf nodes must be placed on the boundaries. A layout so produced is a compromise between a pure "H-tree" layout and a pure "standard" tree layout, in terms of area required and the maximum link length (which determines the worst propagation delay and hence the system clock rate). Such a layout is shown better than the other layout with the same number of boundary leaf nodes because it takes less area and has shorter longest links.

It is possible to extend this layout approach one step further and consider the cases where only a portion (rather than all) of the subtrees are treated as basic building blocks, while the remaining subblocks follow the pure "H-tree" layout. However, these kinds of alternatives are only to investigate different styles of grouping up the subblocks; they provide no advantage as to the area or the number of boundary leaf nodes.

References

- [1] C. A. Mead and L. A. Conway, "Hierarchically Organized Machines," in *Introduction to VLSI Systems*. Reading, MA: Addison-Wesley, 1980, Ch. 8.4.
- [2] H. T. Kung, "Memory Requirements for Balanced Computer Architectures," *Proc. 18th Annu. Int'l Symp. Computer Architecture*, June 1986, pp. 49-54.
- [3] M. C. Howells and V. K. Agarwal, "A Reconfiguration Scheme for Yield Enhancement of Large Area Binary Tree Architectures," *IEEE Trans. Computers*, vol. C-37, pp. 463-468, Apr. 1988.
- [4] R. P. Brent and H. T. Kung, "On the Area of Binary Tree Layouts," *Information Processing Letters*, vol. 11, pp. 46-48, Aug. 29, 1980.
- [5] E. Horowitz and A. Zorat, "The Binary Tree as an Interconnection Network: Applications to Multiprocessor Systems and VLSI," *IEEE Trans. Computers*, vol. C-30, pp. 247-253, Apr. 1981.
- [6] D. Gordon, "Efficient Embeddings of Binary Trees in VLSI Arrays," *IEEE Trans. Computers*, vol. C-36, pp. 1009-1018, Sept. 1987.
- [7] M. J. Fischer and M. S. Paterson, "Optimal Tree Layout," *Proc. 12th Symp. Theory of Computing*, Apr. 1980, pp. 177-189.